SUMMER INTERNSHIP REPORT

On

"Study of Indigenous Flight Controller using SHAKTI Yamuna RISC-V32 SoC"

By

Akshit Kumar

B.Tech, Electrical Engineering, 2024-2028

Indian Institute of Technology, Ropar

Carried out at

Semi-Conductor Laboratory, Mohali

For

The duration of 4 weeks (25th May– 19th June 2026)

Under the supervision of

Dr. RAJESH K. SRIVASTAVA

Senior Scientist

Analog & Mixed Design Division, SCL, Mohali

ACKNOWLEDGMENT

I would like to express my sincere gratitude and appreciation to all those who have contributed to the completion of this report and for providing me with the invaluable opportunity to undertake a four-week internship at such an early stage in my academic journey.

As a second-year student, this experience has been instrumental in helping me understand the foundational concepts of the industry and their practical applications.

I am particularly thankful to Dr. Rajesh K. Srivastava, Shri Deep Sehgal and everyone in the department, whose encouragement and guidance helped me gain confidence in applying theoretical concepts. Their mentorship allowed me to grasp essential skills and gain meaningful insights into the professional environment.

This internship has significantly enhanced my understanding and appreciation of real-world practices, and I am truly grateful for the hands-on exposure and the learning environment that was both welcoming and enriching.

I would like to express my gratitude to my parents and all the members of SCL, Mohali, for all their kind co-operation and encouragement, which played a crucial role in this internship.

Once again, thank you to everyone involved for their support and belief in my abilities, making this internship a truly rewarding and enriching journey.

Sincerely,
Akshit Kumar
Indian Institute of Technology, Ropar

Abstract

This technical report presents a comprehensive study of porting the ArduPilot open-source autopilot software framework from the ARM (Advanced RISC Machine) Cortex-M4 based STM32F405 microcontroller to the SHAKTI Yamuna System-on-Chip (SoC), an indigenously designed RISC-V (Reduced Instruction Set Computer - Version 5) processor developed by the RISE (Reconfigurable Intelligent Systems Engineering) Group at IIT Madras. This work is conducted under the Ministry of Electronics and Information Technology (MeitY) aimed at establishing a sovereign, open-source processor ecosystem.

The report covers the fundamental principles of Unmanned Aerial Vehicle (UAV) flight dynamics and control, the hardware and software architecture of modern flight controllers, a rigorous comparison between the STM32F405 and the Yamuna C-class SoC, the operational differences in common communication protocols (SPI, I2C, UART, PWM), the architecture of Real-Time Operating Systems (RTOS) including ChibiOS and FreeRTOS, the ArduPilot software stack and its Hardware Abstraction Layer (HAL) porting mechanism and a staged roadmap for producing a functional AP_HAL_SHAKTI implementation.

The primary contribution of this work is the identification and mitigation strategy for six critical technical gaps such as absence of a hardware Floating-Point Unit (FPU), absence of a Direct Memory Access (DMA) controller, lack of a ChibiOS RTOS port for RISC-V, absence of on-chip non-volatile flash memory, differences in interrupt controller architecture and PWM signal jitter and a phased development plan progressing from SDK mastery to a live MAVLink ground station link.

Keywords: *RISC-V, SHAKTI, ArduPilot, UAV, Flight Controller, HAL Porting, FreeRTOS, ChibiOS, STM32F405, DIR-V, ESC, PID Controller, MeitY*

About Semi-Conductor Laboratory (SCL), Mohali

The Semi-Conductor Laboratory (SCL), located in Mohali, Punjab, is an autonomous organization under the Department of Space (DoS), Government of India. Established with the objective of developing indigenous semiconductor technologies, SCL plays a vital role in strengthening India's capabilities in microelectronics, Very Large-Scale Integration (VLSI) design, and semiconductor manufacturing.

SCL is involved in the design, fabrication, testing, and packaging of integrated circuits (ICs) and Micro-Electro-Mechanical Systems (MEMS) devices for strategic, space, defense, and industrial applications. The organization provides end-to-end semiconductor solutions, ranging from ASIC design and fabrication to system-level integration.

Apart from fabrication services, SCL actively supports research and development in emerging areas such as RISC-V based processors, System-on-Chip (SoC) design, radiation-hardened electronics, and embedded systems. The laboratory has significantly contributed to India's self-reliance in semiconductor technology through the development of indigenous processors and specialized electronic systems for critical applications.

SCL also collaborates with academic institutions, research organizations, and industry partners by offering fabrication services, training programs, and internship opportunities, thereby fostering innovation and developing skilled manpower in the semiconductor domain.

Through its continuous efforts in semiconductor research and manufacturing, SCL remains a key pillar in advancing India's vision of technological self-reliance and indigenous electronics development.

Table of Contents

1. Introduction to the SHAKTI Project and Yamuna SoC

1.1 Background and Motivation

India's dependence on foreign microprocessor designs - predominantly ARM-licensed cores manufactured by STMicroelectronics, NXP, Texas Instruments and Qualcomm - represents a strategic vulnerability in national defense, aerospace and critical infrastructure. Every processor sold under an ARM license requires royalty payments to ARM Holdings (now owned by SoftBank) and the underlying Instruction Set Architecture (ISA) cannot be freely inspected, modified, or independently implemented. This proprietary lock-in creates export-control risks and limits India's ability to produce indigenous, verifiable electronic systems for defense and space applications.

The SHAKTI project, initiated in 2014 at the RISE Group of the Indian Institute of Technology Madras (IIT-M), addresses this challenge directly by designing, implementing and taping out processors based on the RISC-V open ISA. RISC-V, originally developed at the University of California, Berkeley, provides a complete, royalty-free, openly standardized instruction set that any organization can implement without licensing fees or legal encumbrances. The Ministry of Electronics and Information Technology (MeitY) of the Government of India has institutionalized this direction through the DIR-V (Design in RISC-V) initiative, of which the Grand Challenge 2025 (GC2025) forms a core research and development programme.

1.2 The SHAKTI Processor Family

The SHAKTI family is organized into six processor classes, each targeting a different performance-power-area (PPA) segment, analogous to the ARM Cortex family:

i. E-class: Ultra-low-power microcontroller, IoT sensors, 100 MHz range.
ii. C-class: Embedded controller with cache, targeting 200 MHz to 1.5 GHz, comparable to ARM Cortex-A35.
iii. I-class: Application processor with out-of-order execution, comparable to ARM Cortex-A57.
iv. M-class: High-performance server processor.
v. S-class: Secure processor with hardware security primitives.
vi. T-class: Deprecated experimental class.

All classes implement the base RISC-V ISA with optional standard extensions. The RISC-V ISA is modular: the base integer instruction set (I) is mandatory and optional extensions are denoted by letters such as M for integer multiplication and division, A for atomic memory operations, C for compressed (16-bit) instructions, F for single-precision floating-point, D for double-precision floating-point, etc.

1.3 The Yamuna SoC and GC2025

Yamuna is the SHAKTI SoC (System-on-Chip) based on the C-class processor core. It implements the RV32IMAC instruction set - 32-bit RISC-V with Integer, Multiply, Atomic and Compressed extensions, but without hardware floating-point extensions. The SoC is provided to GC2025 participants in bitstream form for the Xilinx Artix-7 100T Field-Programmable Gate Array (FPGA) development board.

The key hardware specifications of the Yamuna SoC are summarized in Table 1.1 below.

Parameter	Specifications
Processor Core	SHAKTI C-class, 6-stage in-order pipeline
Instruction Set Architecture (ISA)	RISC-V RV32IMAC (32-bit)
Target Clock Frequency	50–100 MHz on FPGA; ~1.5 GHz in silicon
Main Memory (RAM)	256 MB DDR3 SDRAM (off chip)
Boot Memory	8 KB Boot ROM (on-chip)
Non-Volatile Storage	External SPI Quad-Flash (16 MB on Artix-7 board)
UART (Universal Asynchronous RX/TX)	3 instances
SPI (Serial Peripheral Interface)	2 instances
I2C (Inter-Integrated Circuit)	2 instances
GPIO (General Purpose Input/Output)	32 pins
PWM (Pulse Width Modulation)	6 channels
GPT (General Purpose Timer)	4 channels
Interrupt Controller	PLIC (Platform Level Interrupt Controller) + CLINT
Analog-to-Digital Converter (ADC)	XADC (Xilinx on-board, 12-bit)
Ethernet	Ethernet Lite (10/100 Mbps, Xilinx IP)
Hardware FPU	Not present in RV32IMAC
DMA Controller	Not present in current Yamuna SoC
FPGA Target	Xilinx Artix-7 100T (xc7a100tcsg324)

Table 1.1: Yamuna SoC Hardware Specifications (GC2025)

1.4 Significance for Indigenous Defense and Aerospace Applications

A processor with full, open IP ownership carries profound implications for national security applications. When the processor ISA, RTL (Register Transfer Level) description and firmware toolchain are all open-source and indigenously developed, a government agency or defense establishment can:

- Formally verify the processor's behavior against its specification, ruling out hardware backdoors.
- Customize the pipeline or add application-specific instructions without ARM licensing constraints.
- Manufacture the chip domestically under MeitY's semiconductor fab programme without export-control implications.
- Maintain the full supply chain under domestic control, eliminating geopolitical risk.

The GC2025 programme specifically targets autonomous systems such as UAVs, ground robots and marine craft: as priority application domains, making a RISC-V flight controller a strategically significant deliverable.

2. SHAKTI SDK, RISC-V Architecture and Board Support Package

2.1 The RISC-V Instruction Set Architecture

RISC-V (pronounced 'risk five') is an open, free standard Instruction Set Architecture (ISA) based on established Reduced Instruction Set Computing (RISC) principles. Unlike proprietary ISAs such as x86 (Intel/AMD) or ARM Thumb-2, the RISC-V specification is published under a Creative Commons license and may be freely implemented by any party without royalty or license fee.

RISC-V follows a load-store architecture: arithmetic and logic operations operate only on registers and data is transferred between registers and memory exclusively through explicit load and store instructions. The base integer ISA (RV32I) defines 32 general-purpose registers, each 32 bits wide (x0 through x31), where x0 is hardwired to zero. Instructions are fixed 32 bits wide in the base set, with optional 16-bit compressed encoding (C extension) allowing denser code.

> **ISA String Interpretation:** RV32IMAC = 32-bit base (RV32) + Integer (I) + Multiply/Divide (M) + Atomic (A) + Compressed (C)

The absence of the F (single-precision float) and D (double-precision float) extensions in RV32IMAC is the most consequential hardware limitation for a flight controller application, as discussed in detail in Section 9.

2.2 RISC-V Privilege Architecture and Trap Mechanism

RISC-V defines three privilege levels: Machine mode (M-mode, highest), Supervisor mode (S-mode) and User mode (U-mode). On the Yamuna SoC running bare-metal or FreeRTOS, only M-mode is used. All interrupt and exception handling occurs through the trap mechanism: when an interrupt fires or a fault occurs, the hardware atomically saves the program counter to the MEPC (Machine Exception Program Counter) CSR (Control and Status Register), saves the cause code to MCAUSE and jumps to the address stored in MTVEC (Machine Trap Vector).

The software trap handler must save all 32 registers to the active stack, determine the interrupt source from MCAUSE, dispatch to the appropriate handler, restore registers and execute MRET to return. This software save/restore requirement (versus ARM Cortex-M's hardware stack push of a subset of registers) adds approximately 20–40 cycles of interrupt latency and is a key consideration when implementing the ISR-driven UART ring buffer (Section 8.3.2).

2.3 The SHAKTI Software Development Kit (SDK)

The SHAKTI SDK (Software Development Kit) is the Board Support Package (BSP) provided for the Yamuna SoC. A BSP is a collection of low-level software components that provides the minimum environment needed to develop and run applications on a specific hardware platform. The SHAKTI SDK comprises the following subsystems:

- Peripheral Drivers: C-language functions wrapping direct register accesses for each hardware peripheral (UART, SPI, I2C, GPIO, PWM, PLIC, CLINT). These drivers encapsulate the base addresses and register layouts defined in the SoC's memory map.

- Startup Code: Assembly-language routines that execute at reset before main () is called: initializing the stack pointer, zeroing the BSS (Block Started by Symbol) segment, copying initialized data from flash to RAM and calling the C runtime entry point.
- Linker Script: A text file directing the linker to place code (text segment) at DDR3 base address 0x80000000, stack at the top of DDR3 and global data in the appropriate sections.
- Build System: A GNU Make-based build system producing ELF (Executable and Linkable Format) binaries.
- Debug Interface: OpenOCD (Open On-Chip Debugger) configuration for JTAG debugging via the on-board FTDI (Future Technology Devices International) USB-to-JTAG bridge.

2.4 Memory Map and Peripheral Addressing

In the RISC-V architecture, peripherals are not accessed through a separate I/O address space (as in x86) but are memory-mapped: each peripheral register occupies a fixed address in the same address space as RAM and ROM. Writing to address 0x00011300 transmits a byte through UART0; reading from address 0x00020008 reads the SPI0 status register. This is identical in concept to STM32's memory-mapped peripheral model, though the base addresses differ entirely. The key regions of the Yamuna memory map are shown in Table 2.1.

Peripheral	Base Address	Description
UART0	**0x00011300**	Console / debug serial output
UART1	**0x00011400**	GPS or RC receiver input
UART2	**0x00011500**	MAVLink telemetry radio
SPI0	**0x00020000**	Primary IMU bus
SPI1	**0x00021000**	Secondary sensor bus
I2C0	**0x00030000**	Barometer / Compass
I2C1	**0x00031000**	Secondary I2C
GPIO	**0x00040100**	32 general-purpose pins
PWM0–PWM5	**0x00050000+**	Motor / servo output
PLIC	**0x0C000000**	Platform Level Interrupt Controller
CLINT	**0x02000000**	Core Local Interrupt (timer + SW)
DDR3 RAM	**0x80000000**	256 MB main memory: program runs here

Table 2.1: Yamuna SoC Memory Map (Selected Peripherals)

3. UAV Fundamentals: Aerodynamics and Flight Mechanics

3.1 Definition and Classification of Unmanned Aerial Vehicles

An Unmanned Aerial Vehicle (UAV), colloquially termed a 'drone,' is an aircraft that operates without a human pilot on board. UAVs are classified by their aerodynamic configuration into fixed-wing (conventional aircraft geometry), rotary-wing (helicopters, multicopters) and hybrid types. The most prevalent configuration in commercial and research applications is the multirotor: specifically, the quadrotor (four rotors): due to its mechanical simplicity, hover capability and agile maneuverability. This report focuses exclusively on the quadrotor configuration.

3.2 Quadrotor Flight Mechanics

A quadrotor generates lift and control moments exclusively through the speed variation of four electrically driven propellers. Each propeller produces a thrust force directed vertically upward (assuming a levelled frame) and a reactive torque about the vertical axis due to aerodynamic drag. The four propellers are arranged in two contra-rotating pairs to cancel net torque in hover:

- Motors 1 and 3 (diagonal pair): rotate counterclockwise (CCW)
- Motors 2 and 4 (diagonal pair): rotate clockwise (CW)

The four primary control degrees of freedom (DoF) are achieved as follows:

Control	Axis	Mechanism
Throttle	Vertical (heave)	All four motors increase or decrease equally: net thrust vs. weight determines climb/descent.
Roll	X-axis (lateral tilt)	Motors on one side increase; opposite side decreases: differential thrust creates roll moment.
Pitch	Y-axis (fore/aft tilt)	Front pair increases: rear pair decreases (or vice versa): differential thrust creates pitch moment.
Yaw	Z-axis (heading rotation)	CW pair spins faster than CCW pair (or vice versa): imbalanced reactive torques create yaw.

Table 3.1: Quadrotor Control Degrees of Freedom

> **Equilibrium Condition:** Total Thrust (4 × T) = Vehicle Weight (mg). All four motors equal → pure hover with zero angular velocity.

3.3 State Variables and Sensor Coverage

A flight controller must maintain continuous estimates of the vehicle's state vector, which includes position (x, y, z), velocity (ẋ, ẏ, ż), orientation (roll φ, pitch θ, yaw ψ) and angular rates (p, q, r). Each state variable requires one or more sensors for observability:

State Variable	Primary Sensor	Operating Principle
Roll, Pitch (absolute)	Accelerometer	Gravity vector measured in body frame; atan2 gives attitude angles.
Angular rates (p, q, r)	Gyroscope (MEMS)	Coriolis effect in vibrating MEMS structure; output in °/s.
Yaw (heading)	Magnetometer (compass)	Earth's magnetic field vector; requires calibration for hard/soft iron.
Altitude (z)	Barometric pressure sensor	Atmospheric pressure decreases ~12 Pa per metre; accuracy ±1 m.
Position (x, y)	GNSS receiver (GPS)	Time-of-flight from satellite constellation; accuracy ±2.5 m CEP.
Velocity (ẋ, ẏ, ż)	GPS Doppler + IMU integration	GPS provides absolute velocity; IMU integration for short-term smoothing.
Altitude (z, precise)	LIDAR / optical flow	Time-of-flight laser or visual tracking; accuracy ±2 cm.

Table 3.2: Vehicle State Variables and Corresponding Sensors

3.4 Sensor Fusion

No single sensor provides a complete, noise-free estimate of all state variables. The accelerometer is accurate in steady state but noisy under vibration. The gyroscope is fast and low noise but drifts over time (integration of white noise accumulates error). Sensor fusion algorithms combine multiple sources to obtain estimates superior to any individual sensor. The most widely implemented algorithms in ArduPilot are:

- Complementary Filter: Blends gyroscope rate integration with accelerometer attitude estimate using a tunable coefficient α (typically 0.98). Simple, low computational cost, adequate for most applications.
- Extended Kalman Filter (EKF): Non-linear probabilistic estimator that tracks the full state vector including position, velocity, attitude and biases. Used in ArduPilot's EKF3 for GPS-fused navigation. Computationally intensive: requires floating-point arithmetic at each 400 Hz iteration.

4. UAV Control Systems: Flight Controller, ESC and Sensor Architecture

4.1 The Flight Controller

A flight controller (FC) is a printed circuit board (PCB) integrating a microcontroller (MCU), an IMU (Inertial Measurement Unit) soldered directly to the board (to minimize sensor latency) and connectors for external peripherals. The flight controller firmware (ArduPilot, Betaflight, PX4) reads sensor data, runs attitude and position estimation algorithms, executes the PID control loops and outputs PWM signals to ESCs. The 'flight controller' hardware product and the autopilot firmware are distinct: hardware provides the processor and sensors; firmware provides the intelligence.

> **Key Insight:** Porting ArduPilot to SHAKTI Yamuna turns the GC2025 FPGA board into a flight controller. The hardware is the Arty7 board; the firmware is ArduPilot + AP_HAL_SHAKTI.

4.2 The PID Control Loop

The Proportional-Integral-Derivative (PID) controller is the fundamental feedback control algorithm used in all modern flight controllers. Given a desired state (setpoint, e.g. 0° roll) and a measured state (e.g. current roll angle from IMU), the PID controller computes a corrective output (e.g. motor speed differential) according to:

$$u(t) = K_p \cdot e(t) + K_i \cdot \int e(t)dt + K_d \cdot de(t)/dt$$

Where e(t) is the error (setpoint minus measurement), Kp is the proportional gain, Ki is the integral gain and Kd is the derivative gain. ArduPilot runs this loop at 400 Hz (2.5 ms period), which sets the fundamental real-time deadline for the flight controller firmware.

4.3 Electronic Speed Controller (ESC)

An Electronic Speed Controller (ESC) is a power electronic device that receives a throttle command signal from the flight controller and converts it into three-phase alternating current (AC) to drive a brushless DC (BLDC) motor at the commanded speed. Each motor requires a dedicated ESC. The ESC contains its own microcontroller running firmware (BLHeli_32 or AM32) that implements sinusoidal or trapezoidal commutation of the motor windings.

4.3.1 ESC Signal Protocols

The communication protocol between the flight controller and the ESC defines latency, precision and feature set. The main protocols, in order of increasing sophistication, are:

Protocol	Signal Type	Update Rate	Yamuna Support
PWM 50 Hz	Analog pulse 1000–2000 µs	50 Hz	✓ **Full support**

Oneshot125	Analog pulse 125–250 µs	Up to 2 kHz	**Partial (jitter risk)**
DShot 300	Digital 300 kbit/s packet	Up to 8 kHz	**✗ Requires DMA**
DShot 600	Digital 600 kbit/s packet	Up to 8 kHz	**✗ Requires DMA**
Bidirectional DShot	Digital + RPM telemetry	Up to 8 kHz	**✗ Requires DMA**

Table 4.1: ESC Signal Protocols and Yamuna Compatibility

4.4 Signal Chain Architecture

The complete signal chain from sensor to motor, as implemented in ArduPilot, is illustrated in Figure 4.1 below. This diagram represents the data and control flow that the porting effort must replicate on the Yamuna SoC.

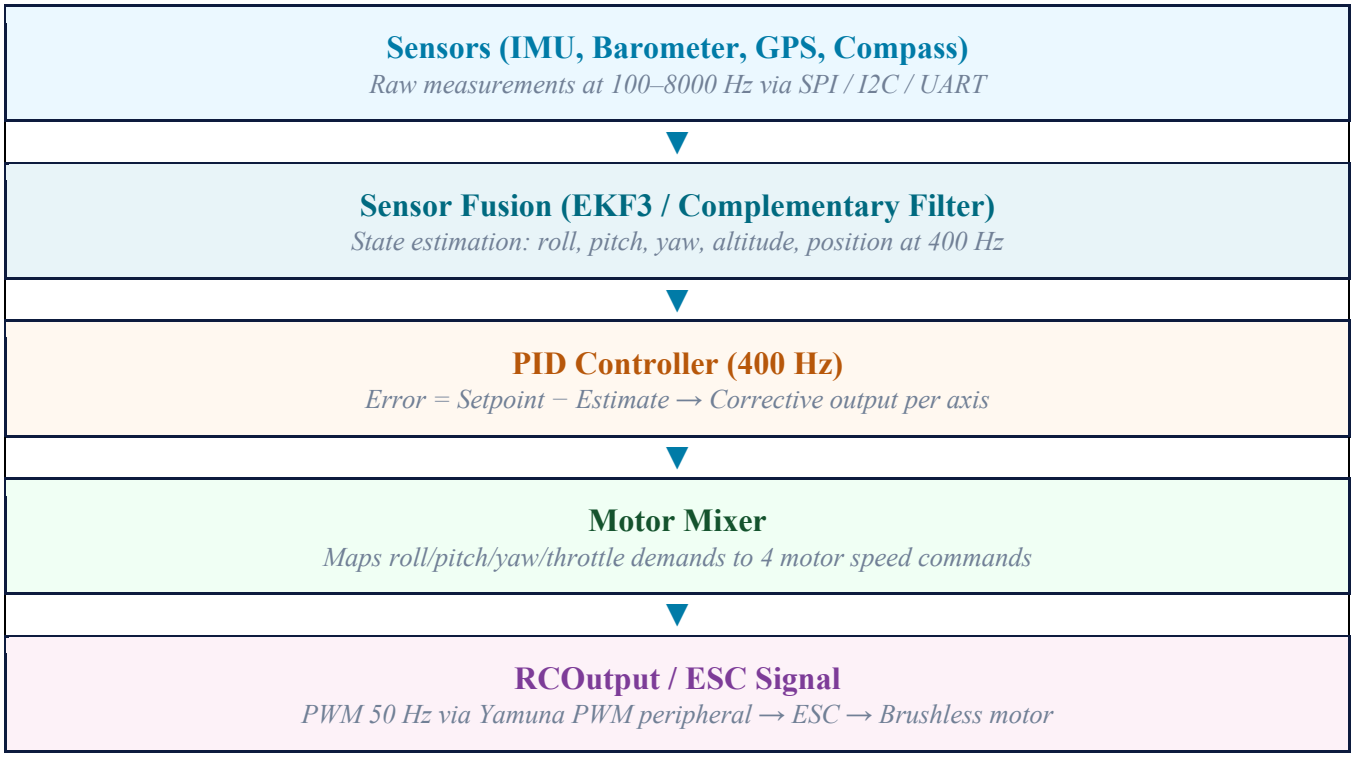

Figure 4.1: *ArduPilot Signal Chain: Sensor to Motor*

5. Comparative Analysis: STM32F405 vs SHAKTI Yamuna C-class

5.1 Rationale for Comparison

The STM32F405 (manufactured by STMicroelectronics) is the most widely deployed microcontroller in open-source flight controllers. It powers the Pixhawk 1, CUAV X7, Matek F405-Wing, OmnibusF4, SpeedyBeeF405 and dozens of other ArduPilot-compatible boards. Its adoption is the direct result of its mature ecosystem, hardware FPU, DMA infrastructure and the availability of ChibiOS as a production-grade RTOS. Understanding its exact capabilities at the register level is a prerequisite for identifying the porting gaps when targeting the Yamuna SoC.

5.2 Full Specification Comparison

Dimension	STM32F405	Yamuna C-class	Verdict
ISA / Architecture	ARM Cortex-M4 (Thumb-2) Closed, proprietary, royalty-bearing	RISC-V RV32IMAC Open, royalty-free, fully inspectable	**✓ Yamuna**
Clock Speed	168 MHz in silicon ART accelerator, 0-wait flash access	50–100 MHz on FPGA ~1.5 GHz in real silicon (tape-out)	**● STM32 (now)**
Hardware FPU	Single-precision IEEE-754 FPU + DSP multiply-accumulate instructions	None (RV32IMAC has no F/D extension) All float ops in software library	**✗ Critical gap**
SRAM / RAM	192 KB on-chip SRAM (single-cycle) 64 KB CCM + 128 KB general purpose	256 MB DDR3 off-chip Higher latency (~10–50 cycles/access)	**~ Trade-off**
Program Flash	1 MB on-chip flash (single-cycle) ArduPilot requires ~1 MB minimum	External 16 MB SPI flash only 8 KB boot ROM for bootloader only	**● Gap (work needed)**
DMA Controller	2 DMA controllers, 16 streams SPI/UART/I2C all DMA-capable	No DMA in current Yamuna SoC All transfers CPU-polled (blocking)	**✗ Critical gap**
Timers / PWM	17 timers at 168 MHz resolution 12×16-bit + 2×32-bit timers	6 PWM channels + 4 GPT channels Adequate for 50 Hz standard PWM	**● Partial**
Interrupt Controller	NVIC: 256 priority levels Hardware nested preemption (CMSIS)	PLIC + CLINT (RISC-V standard) Software-managed priorities, no nesting	**● Different model**

RTOS (ArduPilot)	ChibiOS/RT (native, mature) Full DMA + thread support	No ArduPilot RTOS port exists FreeRTOS RISC-V port to be used	**✗ Must build**
ArduPilot HAL	AP_HAL_ChibiOS (30+ boards) Mature, production-tested	AP_HAL_SHAKTI does not exist Must be written from scratch	**✗ Must build**
IP Sovereignty	ARM licensee (ST-owned) Export-control risk, royalties	Fully open, IIT-Madras developed No royalties, DIR-V aligned	**✓ Yamuna wins**

Table 5.1: STM32F405 vs Yamuna C-class Full Specification Matrix

> **Note:** *Color coding: Green = Yamuna advantage. Amber = gap requiring mitigation. Red = critical gap requiring significant engineering effort.*

5.3 Performance Implications for a Flight Controller

The two most critical gaps from Table 5.1 for the flight controller application are the absence of a hardware FPU and the absence of DMA. Together, these two gaps determine whether ArduPilot's 400 Hz main loop can execute within its 2.5 ms deadline on the Yamuna SoC.

A single-precision floating-point multiply on the STM32F405's FPU executes in 1 clock cycle at 168 MHz. The equivalent operation on Yamuna, emulated by the software float library (libgcc), requires approximately 20–60 clock cycles at 100 MHz on FPGA. For a PID loop with 50 float operations per axis across 4 axes, this represents an increase from approximately 300 ns (STM32 FPU) to approximately 300 µs (Yamuna soft-float at FPGA speed): still within the 2.5 ms budget but leaving far less margin for sensor reads and communication handling.

With DMA absent, the CPU must poll during every SPI byte transfer. An MPU-6000 IMU burst read of 14 bytes at 10 MHz SPI clock takes approximately 11 µs of pure transfer time; without DMA, the CPU is blocked for those 11 µs. At 500 Hz IMU rate, this accounts for 5.5 ms of CPU time per second: approximately 0.55% of a 100 MHz CPU, which is manageable but must be budgeted carefully alongside UART receive ISRs and PWM update tasks.

6. Protocol-Level Differences: SPI, I2C, UART and PWM

A fundamental misconception in embedded systems porting is that two chips 'having SPI' implies interchangeable SPI implementations. The wire-level electrical protocol is indeed standardized (CPOL, CPHA, MOSI, MISO, SCK signals), but every aspect beneath the wire: register layout, DMA integration, FIFO depth, interrupt model, clock configuration and transfer initiation mechanism: differs entirely between STM32 and Yamuna. This section documents each protocol's differences in engineering detail and identifies the mitigation strategy for each.

6.1 SPI: Serial Peripheral Interface

6.1.1 STM32F405 SPI Architecture

The STM32F405 provides three SPI instances (SPI1, SPI2, SPI3) clocked from the APB1 or APB2 bus. The peripheral is configured through two 16-bit control registers (CR1, CR2). CR1 contains the CPOL and CPHA polarity/phase bits, the BR [2:0] baud rate prescaler (dividing the bus clock by 2 to 256), the MSTR master mode bit and the SPE enable bit. CR2 contains the critical TXDMAEN and RXDMAEN bits: when set, these cause the SPI peripheral to issue DMA transfer requests whenever the transmit buffer empties or the receive buffer fills. A single DMA stream can then automatically move an entire burst of bytes between memory and the SPI data register without any CPU involvement. The CPU initiates the transfer by configuring the DMA stream and setting the SPE bit; the DMA interrupt fires only when the complete transfer is finished.

6.1.2 Yamuna SPI Architecture

The Yamuna SPI peripheral (SPI0/SPI1) provides the same wire-level protocol but through a fundamentally different register model. The SPI Control Register (SPI_CR) contains CPOL, CPHA, SPE (enable) and MSTR bits. The SPI Status Register (SPI_SR) contains the SPIF flag (set when a byte transfer completes) and WCOL (write collision). The SPI Data Register (SPI_DR) is 8-bit only. Critically absent are TXDMAEN, RXDMAEN, FIFO depth bits and any DMA request line. The only transfer mechanism available is CPU polling: the software writes a byte to SPI_DR, polls the SPIF flag until it sets, reads the received byte from SPI_DR and repeats for each byte in the transfer.

Attribute	STM32F405 SPI	Yamuna SPI
Max Speed	42 Mbit/s (SPI1 on APB2)	~10 Mbit/s (limited by FPGA timing)
DMA support	Yes: TXDMAEN/RXDMAEN in CR2	No: no DMA request lines
Transfer initiation	Set TXDMAEN, program DMA stream	Write byte, poll SPIF flag, repeat
CPU load (14-byte IMU burst)	~0 cycles (DMA does it)	~2000 cycles (polled)
Hardware CRC	Yes: CRCPR, RXCRCR, TXCRCR	No

Chip Select (CS)	Hardware NSS (SSOE bit) or GPIO	Manual GPIO only
Instances	3 (SPI1/2/3)	2 (SPI0/SPI1)
Data width	8 or 16 bit (DFF bit in CR1)	8 bit only
FIFO depth	None (single byte register)	None (single byte register)

Table 6.1: SPI Peripheral Comparison: STM32F405 vs Yamuna

> **Mitigation Strategy:** Read all 14 IMU registers in a single SPI burst transaction (one CS assert/deassert cycle, 14 consecutive polled byte exchanges). Reduce IMU sample rate from 8 kHz to 500 Hz. Profile CPU load and confirm loop budget.

6.2 I2C: Inter-Integrated Circuit

6.2.1 STM32F405 I2C: Known Silicon Errata

The STM32F405 I2C peripheral (I2C1/I2C2/I2C3) is notoriously difficult to use correctly. The official ST errata document (DocID022152) explicitly states: 'The I2C peripheral is not fully compliant with the SMBus standard. If a bus error occurs while the peripheral is trying to switch to controller mode by setting the START bit, the Start condition is not properly generated.' The practical manifestation is the 'BUSY flag deadlock': after any failed I2C transaction (bus glitch, NAK, power cycle of a sensor), the BUSY bit in SR2 remains permanently asserted and all further I2C calls return immediately with a BUSY error. Recovery requires setting the SWRST (Software Reset) bit in CR1, then re-initializing the peripheral. ArduPilot's ChibiOS I2C driver contains explicit timeout and SWRST recovery code to handle this, making it significantly more complex than the Yamuna equivalent.

6.2.2 Yamuna I2C Architecture

The Yamuna I2C peripheral implements a simpler open-core Wishbone/AXI-native I2C controller. The key registers are: I2C_PR (prescaler for SCL frequency), I2C_CTL (start/stop/read/write/acknowledge bits), I2C_SR (a single status register containing TIP: Transfer In Progress: RXACK, BUSY and AL flags) and I2C_DR (data register). The BUSY flag is architecturally cleaner, and the errata documented for STM32 does not apply. However, no DMA path exists for I2C; all transfers are CPU-polled using the TIP flag.

Attribute	STM32F405 I2C	Yamuna I2C
Max speed	400 kHz (Fast Mode)	400 kHz (Fast Mode)
Clock config	CCR + TRISE registers + FREQ [5:0]	Single prescaler register
DMA support	Yes (for reliability)	No: CPU polled
Known errata	BUSY flag deadlock: needs SWRST	No equivalent errata

Status registers	SR1 + SR2 (two 16-bit registers)	Single SR register
Instances	3 (I2C1/2/3)	2 (I2C0/I2C1)
Interrupt	ITEVTEN / ITBUFEN in CR2	PLIC interrupt via I2C_IRQ

Table 6.2: I2C Peripheral Comparison: STM32F405 vs Yamuna

> **Mitigation Strategy:** Implement timeout counters on all TIP polling loops (abort after 100,000 cycles). Add I2C bus recovery (9 SCL clock cycles with SDA high) in the error handler. Yamuna's simpler I2C is an advantage here.

6.3 UART: Universal Asynchronous Receiver/Transmitter

The most operationally critical protocol difference for flight controller performance is UART. ArduPilot uses UART for: GPS input at 115200 baud (NMEA or UBX binary), MAVLink telemetry at 57600 baud, RC receiver input (SBUS/CRSF protocols) and debug console output. All of these are receive-heavy applications where data arrives asynchronously and must be buffered without loss.

On the STM32F405, the UART DMA circular mode is the standard implementation: a DMA stream is programmed to continuously move bytes from the USART_DR register into a fixed SRAM ring buffer. The DMA hardware fires a Half-Transfer (HT) interrupt at the midpoint and a Transfer-Complete (TC) interrupt at the end of each buffer cycle, allowing the application to read processed data. Additionally, the UART IDLE line interrupt fires after one character period of bus inactivity: ArduPilot uses this to detect the end of a GPS sentence or MAVLink packet without polling.

On Yamuna, no DMA path exists. The UART RX data register is a single byte; if a second byte arrives before the CPU reads the first, the first byte is overwritten and permanently lost. At 115200 baud, bytes arrive every 87 microseconds. If any interrupt service routine (ISR): particularly the SPI polling loop for the IMU: holds the CPU for more than 87 µs, bytes will be dropped. The mitigation is a software ring buffer filled from within a PLIC-driven UART RX interrupt: the ISR fires on each received byte, reads it from the data register and pushes it into a 512-byte circular buffer in DDR3. The application reads from the ring buffer at leisure.

Attribute	STM32F405 UART	Yamuna UART
RX buffering	DMA circular buffer (automatic)	Software ring buffer (ISR-driven)
RX overrun risk	Only if SRAM buffer full	Any ISR > 87 µs at 115200 baud
Packet detection	IDLE line interrupt (hardware)	Software timer or character count
Flow control	Hardware RTS/CTS (USART1/2/3)	None: software only
Instances	4 USART + 2 UART (6 total)	3 UART

Max baud rate	10.5 Mbit/s (USART1 on APB2)	~1 Mbit/s on FPGA
Sync mode	Available on USART (CK pin)	Async only

Table 6.3: UART Peripheral Comparison: STM32F405 vs Yamuna

> **Mitigation Strategy:** Implement ISR-driven 512-byte ring buffer for each UART. Limit MAVLink baud rate to ≤57600 (174 µs/byte). Ensure SPI ISR duration is bounded below 80 µs. Use CLINT timer to detect UART idle for packet framing.

6.4 PWM: Pulse Width Modulation

On the STM32F405, PWM is generated by the advanced-control timers (TIM1, TIM8) and general-purpose timers (TIM2–TIM5). The timer's counter runs continuously at up to 168 MHz and the output compare unit autonomously toggles the output pin when the counter matches the CCR (Capture/Compare Register) value. This means the PWM signal is completely hardware-generated and independent of CPU activity: even if the CPU is saturated running PID loops, the PWM output maintains perfect timing. DShot protocol is implemented by using the timer in DMA burst mode, where the DMA writes a pre-computed bit pattern of CCR values at the required bit clock rate.

On Yamuna, the PWM peripheral has a period register and a duty-cycle register. The CPU must write the desired duty cycle to the duty register at the start of each PWM cycle. If the CPU is late writing this value (due to interrupt latency or a blocking SPI read), the previous period's duty cycle is repeated, introducing jitter. Additionally, since DShot requires generating bit timings at 300 kbit/s or faster, it is infeasible without DMA: the CPU cannot write individual bit timings to a GPIO register at 300 kHz while simultaneously running PID calculations.

7. Real-Time Operating Systems: RTOS, FreeRTOS and ChibiOS

7.1 The Problem with Bare-Metal Programming

In a bare-metal embedded system, the firmware runs as a single infinite loop with interrupt service routines. The main loop executes tasks sequentially: read IMU, update PID, write PWM, check GPS, parse MAVLink. If any task takes longer than expected, all subsequent tasks are delayed. For a flight controller with a 400 Hz (2.5 ms) deadline, a single blocking SPI read that takes 200 µs causes the PID update to be 200 µs late, which manifests as control jitter and potential instability.

A Real-Time Operating System (RTOS) solves this by providing structured concurrency with deterministic scheduling. An RTOS manages multiple independent tasks (threads), each with its own stack and execution context and enforces priority-based scheduling: a high-priority task will always preempt a lower-priority task within a bounded latency from the moment it becomes ready to run.

> **Definition: Real-Time System:** A system in which the correctness of an output depends not only on the logical result but also on the time at which the result is produced. A flight controller is a hard real-time system: missing a 400 Hz PID deadline is a system failure.

7.2 Core RTOS Concepts

7.2.1 Task (Thread)

An RTOS task is an independently schedulable unit of execution. Each task has a dedicated stack (a region of RAM for its local variables and saved registers), a priority level and a state (running, ready, blocked, or suspended). The RTOS kernel maintains a list of all tasks and selects the highest-priority ready task to run on the CPU at each scheduling decision.

7.2.2 Context Switch

A context switch is the act of saving the CPU state (all general-purpose registers, the program counter and the stack pointer) of the currently running task to that task's Task Control Block (TCB), then loading the saved state of the next task to run. On RISC-V, a context switch saves and restores all 32 general-purpose registers (approximately 128 bytes on RV32). The RISC-V FreeRTOS port implements context switching in assembly language within portasm. S, invoked by the machine software interrupt (MSIP via CLINT). The overhead is approximately 40–60 CPU cycles per context switch.

7.2.3 Preemptive Scheduling

In a preemptive RTOS, the scheduler can suspend a running task at any point in its execution: without the task's cooperation: in order to run a higher-priority task that has become ready. Preemption is triggered by the timer interrupt (the RTOS tick, typically 1 kHz) or by a higher-priority task becoming unblocked (e.g., by an ISR signaling a semaphore). Without preemption (cooperative scheduling), a task runs until it voluntarily yields, which does not provide real-time guarantees.

7.2.4 Synchronization Primitives

Primitive	Analogy	Use in Flight Controller
Mutex	Exclusive key to a shared room	Protecting the SPI bus: only one task reads IMU at a time.
Semaphore (binary)	One-shot notification	ISR signals 'IMU byte ready'; sensor task unblocks to process it.
Semaphore (counting)	Ticket counter for N resources	N UART TX buffer slots available: producer waits if full.
Queue	Message board (FIFO)	Sensor task enqueues IMU readings; PID task dequeues and processes.
Event Group	Bit flags for multiple events	Wait for IMU_READY AND GPS_READY before running EKF update.
Task Notification	Direct-to-task signal	Fastest mechanism: ISR notifies specific task without queue overhead.

Table 7.1: RTOS Synchronization Primitives and Flight Controller Applications

7.2.5 Priority Inversion and Its Mitigation

Priority inversion is a subtle RTOS scheduling pathology: a high-priority task (H) is blocked waiting for a mutex held by a low-priority task (L), but a medium-priority task (M) is preempting L: so H effectively waits behind M, inverting the intended priority order. FreeRTOS mitigates this with priority inheritance: when task H blocks on a mutex held by task L, L's priority is temporarily raised to H's level, allowing L to complete quickly and release the mutex. Once L releases, its priority reverts to normal.

7.3 FreeRTOS: The RTOS for Yamuna

FreeRTOS is a widely deployed, MIT-licensed RTOS kernel designed for resource-constrained microcontrollers. It provides preemptive priority scheduling, mutexes with priority inheritance, counting semaphores, queues, software timers and task notifications. Critically, FreeRTOS provides an official RISC-V 32-bit port (FreeRTOS/Source/portable/GCC/RISC-V/) that implements context switching in RISC-V assembly and uses the CLINT mtime/mtimecmp registers as the RTOS tick timer. This is the RTOS that will replace ChibiOS in the AP_HAL_SHAKTI implementation.

The FreeRTOS configuration file FreeRTOSConfig.h must specify the CPU clock frequency (for tick timer calibration), the tick rate in Hz (1000 for 1 ms tick), maximum task priorities, total heap size for task stacks and the CLINT register addresses for the RISC-V port.

7.4 ChibiOS: The RTOS Used on STM32 by ArduPilot

ChibiOS (developed by Giovanni Di Sirio) is a compact, high-performance embedded RTOS providing two schedulers (RT for real-time, NIL for ultra-minimal) plus a complete Hardware Abstraction Layer (HAL) with drivers for STM32 peripherals. ArduPilot uses ChibiOS/RT on all STM32-based boards through the AP_HAL_ChibiOS layer.

The critical architectural distinction between ChibiOS and FreeRTOS is that ChibiOS uses a fully static memory model: all kernel objects (threads, mutexes, semaphores) are statically allocated at compile time, with no run-time heap allocation in the kernel. This provides absolute determinism but requires knowing the maximum number of threads and synchronization objects at build time. ChibiOS also integrates deeply with the STM32 DMA system: its UART and SPI drivers use DMA by default, falling back to interrupt-driven transfer only when DMA is unavailable.

ChibiOS cannot be ported to RISC-V without a complete rewrite of its RT kernel port layer (portcore.h, chcore.h and the context switch assembly), because the STM32 port makes direct use of ARM Cortex-M architectural features: the NVIC for interrupt prioritisation, the PendSV exception for context switching, the SysTick timer for the RTOS tick and Thumb-2 specific assembly instructions. None of these exist on RISC-V. This is the fundamental reason AP_HAL_ChibiOS cannot be reused for Yamuna and AP_HAL_SHAKTI must be written from scratch.

Feature	ChibiOS/RT	FreeRTOS (RISC-V)
Architecture support	ARM Cortex-M (native), others experimental	ARM + RISC-V (official RV32 port)
Memory model	Fully static: no heap allocation	Configurable: static or dynamic (heap)
Context switch mechanism	PendSV exception (ARM-specific)	MSIP via CLINT (RISC-V standard)
Tick timer	SysTick (ARM Cortex-M specific)	CLINT mtime/mtimecmp (RISC-V standard)
DMA integration	Tightly coupled to STM32 DMA	None built-in: application-managed
ArduPilot support	Native (AP_HAL_ChibiOS, 30+ boards)	No native support: must implement HAL
RISC-V port	Not production-ready	Official, maintained port available
License	GPL v3 (kernel) + Apache 2.0 (HAL)	MIT License

Table 7.2: ChibiOS/RT vs FreeRTOS Comparison

8. ArduPilot Software Stack: Architecture and Layers

8.1 Overview of ArduPilot

ArduPilot is an open-source autopilot software suite supporting fixed-wing aircraft (Plane), multirotor UAVs (Copter), ground vehicles (Rover), submarines (Sub) and helicopters. It is maintained by a global community and is the most capable open-source autopilot available, supporting GPS-guided autonomous missions, terrain following, obstacle avoidance and integration with ground control stations including Mission Planner and QGroundControl via the MAVLink (Micro Air Vehicle Link) telemetry protocol.

The ArduPilot codebase is deliberately structured to separate hardware-specific code from flight logic through a Hardware Abstraction Layer (HAL). This architectural decision, implemented through pure C++ virtual classes, is what makes porting to a new hardware platform feasible: the flight logic layer (PID controllers, EKF, mission planner) is completely portable and never contains hardware-specific code.

8.2 The Complete ArduPilot Software Stack

Figure 8.1 illustrates the complete layered architecture of ArduPilot as it must be implemented for the Yamuna SoC. Each layer communicates only with the layer immediately above and below it.

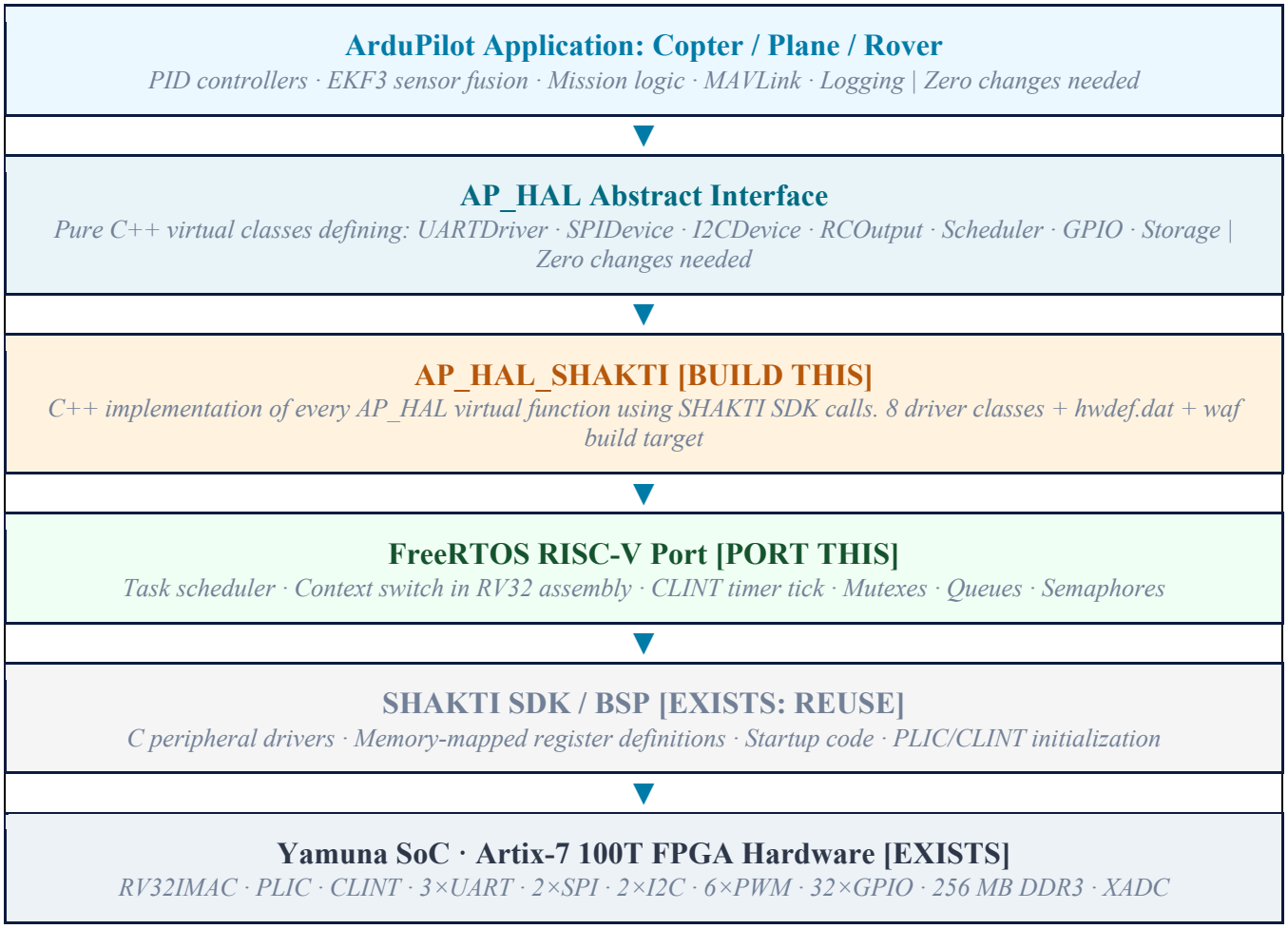

Figure 8.1: *Complete ArduPilot Software Stack for SHAKTI Yamuna: Layer Responsibilities*

8.3 Layer-by-Layer Description

8.3.1 ArduPilot Application Layer

The application layer contains all flight logic. In ArduPilot Copter (the quadrotor firmware), this includes the attitude controller, position controller, EKF3 navigation filter, waypoint mission engine, failsafe system, battery monitoring and MAVLink interface. None of this code references hardware registers, interrupts, or peripheral addresses directly: it accesses hardware exclusively through the AP_HAL interface. This means the entire application layer is compiled and linked identically regardless of whether the target hardware is an STM32H743, a Raspberry Pi, or a SHAKTI Yamuna SoC.

8.3.2 AP_HAL Abstract Interface

AP_HAL defines a set of pure abstract C++ classes, each representing a hardware service. A pure virtual class declares function signatures without implementations: any concrete class inheriting from it must implement every virtual function. The key AP_HAL classes are:

- HAL::UARTDriver :read(), write(), available(), txspace(), begin(baud, rxSize, txSize)
- HAL::SPIDevice :transfer(send, sendLen, recv, recvLen), acquire_bus(), release_bus()
- HAL::I2CDevice :transfer(send, sendLen, recv, recvLen), set_address(addr)
- HAL::RCOutput :write(channel, period_us), enable_ch(channel)
- HAL::Scheduler :delay(ms), delay_microseconds(us), register_timer_process(callback), micros64()
- HAL::GPIO :pinMode(pin, mode), write(pin, value), read(pin)
- HAL::Storage :read_block(dst, src, n), write_block(dst, src, n)
- HAL::AnalogIn :channel(pin), read()

8.3.3 AP_HAL_SHAKTI: The New Implementation

AP_HAL_SHAKTI is the concrete C++ implementation of every AP_HAL abstract class, using SHAKTI SDK function calls as the hardware backend. The directory structure to be created is:

```
ardupilot/libraries/AP_HAL_SHAKTI/    ├── AP_HAL_SHAKTI.h          (master include)    ├──
HAL_SHAKTI_Class.h/cpp   (singleton HAL object)    ├── UARTDriver.h/cpp         (ISR ring
buffer, PLIC-driven)    ├── SPIDevice.h/cpp          (polled burst transfers)    ├──
I2CDevice.h/cpp          (polled + timeout recovery)    ├── RCOutput.h/cpp
(Yamuna PWM register wrapper)    ├── Scheduler.h/cpp          (FreeRTOS tasks + CLINT
micros64)    ├── GPIO.h/cpp               (SHAKTI GPIO driver wrapper)    ├── Storage.h/cpp
(SPI flash parameter store)    ├── AnalogIn.h/cpp           (XADC wrapper)    └──
hwdef/shakti-yamuna/       ├── hwdef.dat            (board description file)       └──
defaults.parm        (default parameter values)
```

8.3.4 hwdef.dat: Board Description File

The hwdef.dat file is a text-based board description that ArduPilot's build system parses to automatically generate C++ header files for the specific hardware configuration. It specifies the MCU type, clock frequency, UART/SPI/I2C bus ordering, IMU type and bus connection, barometer type, compass type,

RC input method, PWM output channels and storage configuration. This file is the primary configuration interface between the board hardware and ArduPilot's device driver selection.

8.3.5 FreeRTOS Integration in the Scheduler

The SHAKTI_Scheduler class is responsible for initializing FreeRTOS, creating all ArduPilot tasks and implementing the AP_HAL timing functions (micros64(), millis(), delay()). The micros64() function reads the CLINT mtime register and divides by the CPU frequency in MHz to convert clock cycles to microseconds. ArduPilot's own AP_Scheduler module calls register_timer_process() to register task functions that the Scheduler calls at 1 kHz from the main ArduPilot FreeRTOS task.

9. Porting Challenges: The Six Critical Technical Gaps

This section documents in detail the six principal technical differences between the STM32F405 and Yamuna SoC that present challenges for the ArduPilot porting effort. For each gap, the severity is assessed, the technical root cause is explained, and concrete mitigation strategies are provided.

9.1 Gap 1: Absence of Hardware Floating-Point Unit (FPU)

Severity: CRITICAL

Problem Statement: The STM32F405's Cortex-M4 FPU executes a single-precision float multiply in 1 clock cycle at 168 MHz. Yamuna's RV32IMAC has no hardware FPU; the compiler links the software float library (libgcc), where a float multiply requires approximately 20–60 clock cycles. For ArduPilot's EKF3 running at 25 Hz with approximately 500 float operations per update, this represents: STM32 ≈ 3 µs per EKF update; Yamuna at 100 MHz ≈ 300 µs per EKF update. The EKF update remains within budget (25 Hz → 40 ms period), but PID loops at 400 Hz (2.5 ms budget) must be carefully profiled.

Mitigation Strategies:

- Use fixed-point arithmetic for all PID loops (scale values by 1000, use 32-bit integer math, convert only at output boundaries).
- Use AP_Math fast approximation functions for sin/cos/atan2 (lookup-table-based, accurate to 0.1%).
- Reduce EKF update rate from 25 Hz to 10 Hz for initial porting; increase once profiling confirms margin.
- Long-term: request an RV32IMACF bitstream from the SHAKTI team to enable single-precision hardware float.
- Compile with -O2 optimization; GCC can often convert simple float sequences to integer equivalents.

9.2 Gap 2: Absence of DMA Controller

Severity: CRITICAL

Problem Statement: ArduPilot's ChibiOS HAL uses DMA for all SPI, UART and I2C transfers. With DMA, peripheral data moves between memory and the peripheral register in hardware while the CPU executes flight code. Without DMA on Yamuna, the CPU must actively participate in every byte transfer. The cumulative CPU load from IMU reads (500 Hz × 14 bytes × ~30 cycles/byte ≈ 210,000 cycles/s), UART receive ISRs (57600 baud × 1 ISR/byte × ~40 cycles/ISR ≈ 2,304,000 cycles/s) and PWM updates (400 Hz × ~500 cycles/update ≈ 200,000 cycles/s) totals approximately 2.7 million cycles per second :2.7% of a 100 MHz CPU. This is manageable but leaves limited margin for EKF and application logic.

Mitigation Strategies:

- IMU SPI: read all 14 sensor registers in a single burst transaction to amortize transaction overhead.
- UART: ISR ring buffer (512 bytes) filled on each received byte; application reads at leisure.
- Lower IMU polling rate from 8 kHz (ArduPilot default) to 500 Hz for initial implementation.
- Verilog extension (stretch goal): implement a single-channel DMA IP block in the RTL. ~100 lines of Verilog, connects to the AXI bus, moves bytes from peripheral register to DDR3 on each interrupt pulse. This permanently resolves the DMA gap and is publishable research.

9.3 Gap 3: No ChibiOS Port for RISC-V

Severity: HIGH

Problem Statement: ArduPilot's entire STM32 HAL (AP_HAL_ChibiOS) depends on ChibiOS/RT, which uses ARM-specific features for context switching (PendSV exception), tick timing (SysTick) and interrupt management (NVIC + CMSIS). None of these architectural features exist on RISC-V. A ChibiOS RISC-V port would require rewriting the entire portcore and chcore assembly files: work equivalent to writing a new RTOS port. Additionally, AP_HAL_ChibiOS's DMA-based UART and SPI drivers are tightly coupled to STM32's DMA system, making code reuse minimal.

Mitigation Strategies:

- Use FreeRTOS's official RISC-V 32-bit port (FreeRTOS/Source/portable/GCC/RISC-V/). Configure FreeRTOSConfig.h for Yamuna's clock and memory.
- Model AP_HAL_SHAKTI on AP_HAL_Linux, which is also non-ChibiOS and uses polling/ISR-based peripheral drivers.
- The ArduPilot porting guide explicitly supports new HAL implementations: no ChibiOS requirement at the application layer.

9.4 Gap 4: No On-Chip Non-Volatile Flash Memory

Severity: HIGH

Problem Statement: ArduPilot stores hundreds of user-configurable parameters (PID gains, RC calibration, sensor offsets, flight mode assignments) in non-volatile memory that survives power cycles. The STM32F405 has 1 MB of on-chip flash with sector-erase and byte-program capabilities, directly accessible at the processor's bus speed. Yamuna has only 8 KB of boot ROM (read-only) and 256 MB of volatile DDR3 RAM. All persistent storage must use the external 16 MB SPI Quad-Flash chip on the Arty-7 board, which requires erase-before-write (4 KB sector granularity), involves SPI bus time for every read and write and must implement wear-levelling to ensure long-term reliability.

Mitigation Strategies:

- Implement AP_HAL_SHAKTI Storage class backed by the external SPI flash, using the upper 64 KB of the flash address space for parameters.
- Implement a sector-based wear-levelling scheme: maintain a write counter per sector; rotate to the next sector when a threshold is reached.
- For development phase: use a RAM-backed storage class (parameters live in DDR3, lost on power cycle). Replace with SPI flash once the rest of the HAL is confirmed working.
- Alternative: implement storage as a FAT filesystem file on an SD card connected via SPI2: directly analogous to AP_HAL_Linux's file-based storage.

9.5 Gap 5: PLIC vs NVIC Interrupt Controller Architecture

Severity: MEDIUM

Problem Statement: The STM32F405's NVIC (Nested Vectored Interrupt Controller) supports 256 hardware priority levels with automatic nested preemption: when a higher-priority interrupt fires during the execution of a lower-priority ISR, the CPU hardware automatically saves the current ISR's state and jumps to the higher-priority handler. This is entirely automatic: no software involvement. The RISC-V PLIC does not support automatic nested interrupts; the software trap handler runs to completion unless the firmware explicitly re-enables interrupts within the trap handler (a non-trivial implementation). Additionally, context switch overhead on RISC-V is higher because software must save all 32 registers versus the hardware-pushed subset on ARM Cortex-M.

Mitigation Strategies:

- Design ISR priority carefully: UART RX ISR (highest PLIC priority), PWM update interrupt (medium), MAVLink TX (low).
- Keep all ISRs short: hardware operations only (push to ring buffer, set flag, complete PLIC claim). No complex logic in ISRs.
- The FreeRTOS RISC-V port handles context switch correctly via the MSIP mechanism: use FreeRTOS task priorities rather than relying on nested PLIC interrupts for scheduling.
- Document latency measurements: measure time from interrupt trigger to first ISR instruction using CLINT mtime timestamps.

9.6 Gap 6: PWM Output Jitter and Absence of DShot Protocol

Severity: MEDIUM

Problem Statement: On STM32, PWM output is generated autonomously by the hardware timer: the CPU sets the CCR register once per period, and the output pin is toggled by the timer hardware with nanosecond precision independent of CPU load. DShot is implemented using timer + DMA burst mode, enabling digital motor commands at 300–1200 kbit/s. On Yamuna, the CPU writes the duty-cycle register at the start of each PWM period; if the write is delayed by interrupt latency, the previous period's value is repeated, introducing jitter in the motor command. At 50 Hz, the jitter must remain below approximately ±50 µs to avoid noticeable motor vibration. DShot is infeasible without DMA: the bit timings at 300 kbit/s require register writes every 3.3 µs, while the CPU is simultaneously processing PID loops.

Mitigation Strategies:

- Dedicate a high-priority FreeRTOS task exclusively to PWM updates; use vTaskDelayUntil() for absolute periodic timing.
- Minimize all ISR durations (SPI reads, UART processing) to reduce sources of jitter.
- Accept 50 Hz standard PWM as the only supported ESC protocol: adequate for all ArduPilot autonomous flight applications.
- Long-term: implement a simple DMA IP in Verilog that can service the PWM peripheral, enabling Oneshot125 or DShot with low CPU load.

10. Porting Roadmap: Phased Development Plan

The porting roadmap is structured as a sequence of seven stages, each with a defined scope, task list and demonstrable deliverable. Stages 0 through 5 constitute the primary internship scope; Stage 6 is a stretch goal representing the highest-impact outcome.

Stage	Title	Timeline	Key Deliverable
Stage 0	**SHAKTI SDK Mastery**	Weeks 1-3	All SHAKTI SDK peripheral APIs exercised and documented.
Stage 1	**FreeRTOS RISC-V Integration**	Weeks 4-6	FreeRTOS running on RISC-V.
Stage 2	**AP_HAL_SHAKTI Skeleton**	Weeks 7-9	waf copter produces a valid ELF binary for the SHAKTI target.
Stage 3	**Driver Implementations**	Weeks 10-14	ArduPilot Copter boot log shows: IMU detected, Barometer detected, Compass detected.
Stage 4	**RCOutput and ESC Arming**	Weeks 15-17	Four motors armed and spinning from Mission Planner command via Yamuna ArduPilot firmware.
Stage 5	**MAVLink Ground Station Link**	Weeks 18-20	Video: SHAKTI Yamuna attitude displayed live in Mission Planner.
Stage 6	**Bench Stabilization Test (Stretch)**	Weeks 21+	First RISC-V indigenous flight controller demonstrably stabilizing a physical UAV frame.

Table 10.1: Porting Roadmap Summary

Stage 0: SHAKTI SDK Mastery

Timeline: Weeks 1-3 | **Goal:** Establish confident bare-metal control of all Yamuna peripherals.

Task List:

- Flash Yamuna bitstream to Artix-7 via Vivado Hardware Manager.
- Install RISC-V toolchain; verify riscv32-unknown-elf-gcc resolves correctly.
- Write Hello World over UART0; verify output in miniterm at 19200 baud.
- GPIO blink test: toggle onboard LED at 1 Hz using gpio_write().
- SPI IMU read: connect MPU-6000; read WHO_AM_I register (expect 0x68).
- SPI burst read: read all 14 sensor registers in one transaction; print raw values.
- I2C sensor: read BMP280 barometer chip-ID register (expect 0x60).
- PWM output: generate 1500 µs pulse at 50 Hz; verify on oscilloscope.
- PLIC/UART ISR: implement 512-byte ring buffer; verify no byte loss at 115200 baud.

Deliverable: *All SHAKTI SDK peripheral APIs exercised and documented. Bare-metal application reads IMU at 500 Hz and outputs data via UART.*

Stage 1: FreeRTOS RISC-V Integration

Timeline: Weeks 4-6 | **Goal:** Establish preemptive multitasking on Yamuna using FreeRTOS.

Task List:

- Clone FreeRTOS repository; locate portable/GCC/RISC-V/ port directory.
- Write FreeRTOSConfig.h: CPU clock 100 MHz, tick 1 kHz, heap 256 KB, 8 priority levels.
- Set configMTIME_BASE_ADDRESS and configMTIMECMP_BASE_ADDRESS to Yamuna CLINT addresses.
- Create two test tasks: Task A blinks LED at 1 Hz (priority 1); Task B prints counter at 5 Hz (priority 2).
- Verify preemption: Task B should always print on time even when Task A is sleeping.
- Implement mutex-protected SPI bus: two tasks attempting SPI; verify no corruption.
- Measure context switch latency using CLINT mtime timestamps.

Deliverable: *FreeRTOS running on RISC-V. Two concurrent tasks verified correct. Context switch latency documented.*

Stage 2: AP_HAL_SHAKTI Skeleton

Timeline: Weeks 7-9 | **Goal:** Create the AP_HAL_SHAKTI directory structure and achieve successful compilation.

Task List:

- Create directory structure under ardupilot/libraries/AP_HAL_SHAKTI/.
- Implement stub versions of all virtual functions (return 0, print error, or no-op).
- Write hwdef.dat: MCU type, clock, UART order, SPI/I2C order, IMU type, PWM outputs.
- Add shakti-yamuna to Tools/ardupilotwaf/boards.py with RV32IMAC toolchain flags.
- Write wscript modelled on AP_HAL_Linux/wscript.
- Run ./waf configure --board shakti-yamuna; resolve all missing symbol errors.
- Run ./waf copter; confirm binary is produced (though it will not yet function).

Deliverable: *waf copter produces a valid ELF binary for the SHAKTI target. Zero linker errors.*

Stage 3: Driver Implementations

Timeline: Weeks 10-14 | **Goal:** Implement all AP_HAL virtual functions with real hardware functionality.

Task List:

- UARTDriver: begin(), read(), write(), available() backed by ISR ring buffer.
- SHAKTI_Scheduler: init() creates FreeRTOS tasks; micros64() reads CLINT mtime; register_timer_process() dispatches 1 kHz callbacks.
- SPIDevice: transfer() performs polled burst read/write with GPIO CS.
- I2CDevice: transfer() implements START-address-data-STOP with timeout recovery.
- RCOutput: write() and enable_ch() configure Yamuna PWM period and duty registers.
- GPIO: pinMode() and write() wrap SHAKTI GPIO SDK functions.

- Storage: initial RAM-backed implementation (replace with SPI flash in Stage 5).
- Load and run ArduPilot Copter; examine UART0 boot log for successful sensor detection messages.

Deliverable: *ArduPilot Copter boot log shows: IMU detected, Barometer detected, Compass detected. All sensors read without error.*

Stage 4: RCOutput and ESC Arming

Timeline: Weeks 15-17 | **Goal:** Achieve motor spin under ArduPilot GCS control.

Task List:

- Verify PWM task runs at 400 Hz with bounded jitter (<50 µs).
- Implement ESC arming sequence: 2 seconds of 1000 µs minimum throttle.
- Connect one test ESC and motor: arm via Mission Planner 'arm' button.
- Verify motor spins at commanded throttle percentage.
- Test all 4 motor channels simultaneously.

Deliverable: *Four motors armed and spinning from Mission Planner command via Yamuna ArduPilot firmware.*

Stage 5: MAVLink Ground Station Link

Timeline: Weeks 18-20 | **Goal:** Demonstrate live attitude telemetry in Mission Planner from the SHAKTI board.

Task List:

- Connect USB-UART adapter to UART2; configure as MAVLink telemetry port.
- Open Mission Planner; connect to SHAKTI serial port at 57600 baud.
- Tilt board by hand; verify artificial horizon in Mission Planner HUD tracks in real time.
- Record video demonstration for submission.

Deliverable: *Video: SHAKTI Yamuna attitude displayed live in Mission Planner. This is the primary CV and report demonstration artefact.*

Stage 6: Bench Stabilization Test (Stretch)

Timeline: Weeks 21+ | **Goal:** Demonstrate active attitude stabilization on a physical quadrotor frame.

Task List:

- Mount SHAKTI board on quadrotor frame with 4 motors and ESCs.
- Secure to a test jig that allows tilt but prevents free flight.
- Tune PID gains; verify that tilt perturbations are actively corrected by differential motor response.

Deliverable: *First RISC-V indigenous flight controller demonstrably stabilizing a physical UAV frame.*

11. Conclusion and Next Steps

11.1 Summary of Findings

This report has presented a comprehensive technical analysis of the challenge and opportunity of porting the ArduPilot autopilot framework from the ARM STM32F405 microcontroller to the SHAKTI Yamuna RV32IMAC System-on-Chip. The following principal findings are established:

- The SHAKTI Yamuna SoC is architecturally capable of hosting a full ArduPilot flight controller. The 256 MB DDR3 memory, 3 UART interfaces, 2 SPI buses, 2 I2C buses, 6 PWM channels and 32 GPIO pins provide sufficient peripheral resources for a functional quadrotor autopilot.
- The six critical gaps: no hardware FPU, no DMA controller, no ChibiOS RISC-V port, no on-chip flash, different interrupt controller model and PWM jitter, are all mitigatable at the software level for an initial functional port, though they represent engineering effort and performance trade-offs relative to the STM32F405.
- The AP_HAL porting architecture is well-suited to this work. The strict separation between the ArduPilot application layer and its Hardware Abstraction Layer means the flight logic is completely reusable; only the eight AP_HAL driver classes must be newly written.
- FreeRTOS's official RISC-V 32-bit port provides a production-quality RTOS foundation, replacing ChibiOS's role on the SHAKTI platform.
- The standard PWM 50 Hz ESC protocol is fully supported by Yamuna's PWM peripheral and is sufficient for all ArduPilot autonomous flight applications including GPS-guided missions, altitude hold and position hold.
- The national significance of this work aligns directly with MeitY's DIR-V initiative: a demonstrated ArduPilot port on an indigenous processor establishes a sovereign, open-source UAV control stack with no ARM royalties, no proprietary ISA and no export-control dependencies.

11.2 Recommended Next Steps

Following the completion of the primary internship scope (Stages 0–5), the following technical directions are recommended for continuation by the next project phase:

- Tape-out of SHAKTI C-class with the RV32IMACF extension (adding hardware single-precision FPU): Eliminates the most significant performance gap and enables full ArduPilot EKF3 at rated 25 Hz update without fixed-point workarounds.
- RTL implementation of a minimal DMA IP block in Verilog (AMBA AXI-Lite compatible, single channel): Resolves the DMA gap for SPI and UART, enabling higher IMU sample rates and reliable high-baud-rate UART reception.
- AM32 ESC firmware port to RISC-V: Since AM32 is open-source and targets 32-bit MCUs, a RISC-V port would enable an entirely indigenous ESC firmware, completing the full indigenous stack from processor to motor.

- ChibiOS RISC-V port research: A minimal ChibiOS/RT kernel port to RISC-V would enable direct reuse of AP_HAL_ChibiOS's production-tested DMA infrastructure, significantly reducing porting effort for future ArduPilot updates.
- Integration with Aakash (SHAKTI application processor) for onboard Linux running MAVProxy as a companion computer: Enabling advanced autonomy features (computer vision, ROS integration) while the Yamuna handles real-time flight control.

11.3 Final Assessment

The technical challenge of porting ArduPilot to the SHAKTI Yamuna SoC is significant but tractable within the scope of a dedicated internship. The work produces measurable, demonstrable deliverables at each stage - from a UART print at Stage 0 to a live Mission Planner attitude display at Stage 5, providing continuous progress signals. More importantly, the work constitutes a genuine contribution to India's sovereign embedded systems ecosystem: the first demonstrated instance of a fully open-source flight controller stack, from instruction set to autopilot firmware, running on an indigenously designed processor.

References

[1] SHAKTI Processor Project, IIT Madras RISE Group. SHAKTI GC2025 SDK User Manual. GitLab: gitlab.com/shaktiproject/gc2025, 2024.

[2] SHAKTI Project. SHAKTI SoC User Manual, v2.1. Available: shakti.org.in/docs/shakti-soc-user-manual.pdf, 2024.

[3] ArduPilot Development Team. ArduPilot Porting Guide. Available: ardupilot.org/dev/docs/porting.html, 2025.

[4] ArduPilot Development Team. AP_HAL Architecture. Available: ardupilot.org/dev/docs/ap-hal.html, 2025.

[5] RISC-V Foundation. The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA, Document Version 20191213. Available: riscv.org/specifications/, 2019.

[6] Patterson D., Waterman A. The RISC-V Reader: An Open Architecture Atlas. Strawberry Canyon, 2017.

[7] STMicroelectronics. STM32F405/415/407/417 Reference Manual (RM0090). Rev 19. Available: st.com, 2023.

[8] STMicroelectronics. STM32F405xx Errata Sheet (ES0182). Rev 11. Available: st.com, 2023.

[9] Barry R. (FreeRTOS). FreeRTOS Reference Manual: API Functions and Configuration Options. Available: freertos.org, 2022.

[10] FreeRTOS. RISC-V Ports. Available: freertos.org/RTOS-RISC-V.html, 2024.

[11] Di Sirio G. ChibiOS/RT Reference Manual 6.0. Available: chibios.org, 2023.

[12] Mahony R., Hamel T., Pflimlin J.-M. Nonlinear Complementary Filters on the Special Orthogonal Group. IEEE Transactions on Automatic Control, 53(5), 2008.

[13] MeitY. DIR-V (Design in RISC-V) Grand Challenge 2025 Programme Guidelines. Ministry of Electronics and Information Technology, Government of India, 2024.

[14] Memsic. MPU-6000 Product Specification Rev 3.4. InvenSense (TDK), 2013.

[15] Xilinx Inc. 7 Series FPGAs Data Sheet: Overview (DS180). Available: xilinx.com, 2020.